

LET'S
ROCK

THE
FUTURE

FIAP

GRADUAÇÃO

CURSOS PRESENCIAIS E ON-LINE

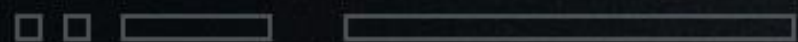
Prof. Dr. Marcus Grilo

Prof. Dr. Marcus Grilo

Ciência da Computação

Computer Organization and Architecture

Organização da Memória



Prof. Dr. Marcus Grilo



<https://www.linkedin.com/in/marcus-grilo/>



profmarcus.grilo@fiap.com.br

Objetivo

Ao final da aula, você deverá ser capaz de:

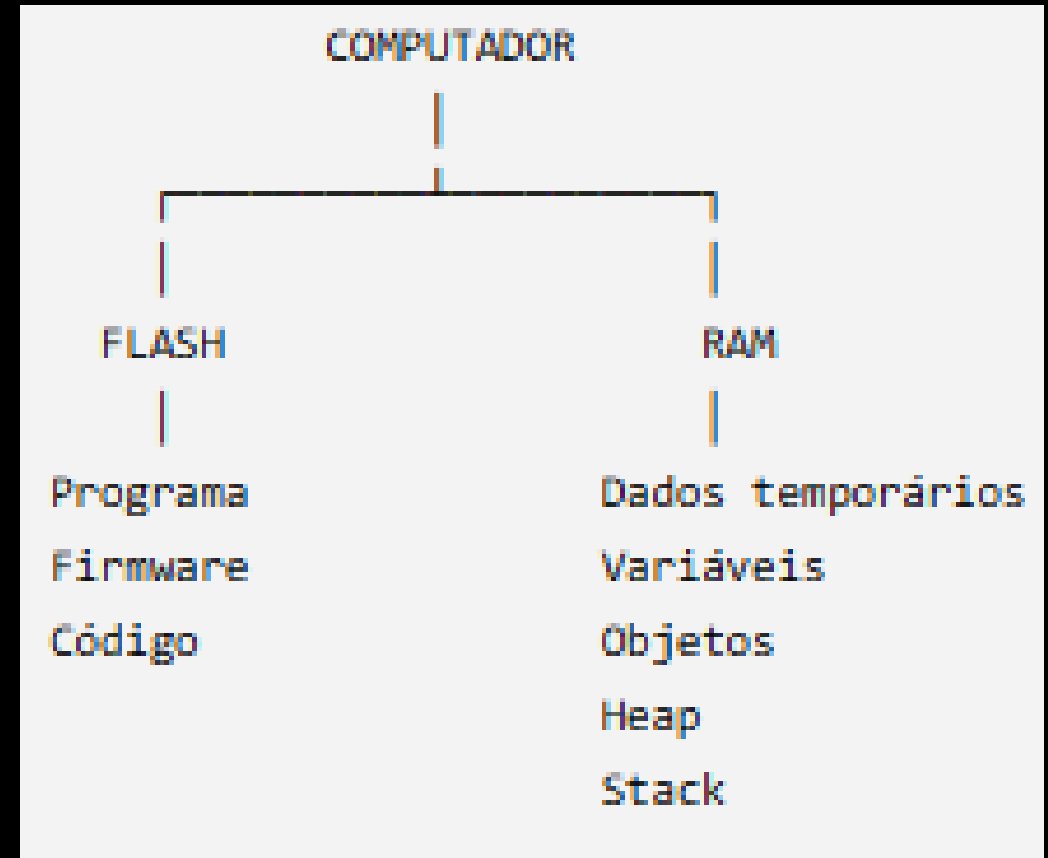
- Organização da memória de um sistema embarcado
- RAM × Flash
- Heap
- Stack
- Variáveis e objetos em memória
- Alocação e liberação de memória
- `gc.mem_alloc()` e `gc.mem_free()`
- Visualização da memória durante a execução

O que o processador realmente faz?

Quando eu escrevo numero = 100, onde o número 100 fica armazenado?

"Mas em qual memória?"

Memória não é uma coisa única.



RAM × Flash

Característica	Flash	RAM
Uso principal	Programa/Firmware	Dados durante execução
Volátil?	Não	Sim
Mantém dados sem energia?	Sim	Não
Velocidade	Menor que registradores/cache	Alta
Variáveis durante execução	Não é o local principal	Sim
Exemplo no Pico	Programa gravado	SRAM

Exemplo

`</>` Python

```
print("Hello World")
```

o código do programa precisa ser armazenado permanentemente no dispositivo.

No Pico, o firmware e o sistema de arquivos ficam associados à memória Flash.

Durante a execução, porém, variáveis e objetos utilizados pelo programa ocupam RAM.

Heap e Stack

Stack

Utilizada principalmente para controlar a execução das funções.

Por exemplo:

```
def soma(a, b):  
    resultado = a + b  
    return resultado
```

```
x = soma(10, 20)
```

Conceitualmente, quando `soma()` é chamada, o processador precisa manter informações relacionadas àquela chamada.

Heap e Stack

Heap

É utilizado para armazenamento dinâmico de objetos.

Por exemplo:

```
lista = [10, 20, 30, 40, 50]
```

ou:

```
nome = "Raspberry Pi Pico"
```

O MicroPython precisa reservar espaço na RAM para esses objetos.

Prática 1 — Medindo a memória

Projeto **Raspberry Pi Pico + MicroPython** no Wokwi.
Podem utilizar apenas o Pico, sem componentes externos.

Código inicial

```
import gc
import micropython

print("=====")
print("      ORGANIZAÇÃO DA MEMÓRIA")
print("=====")

print()

print("RAM livre:")
print(gc.mem_free(), "bytes")

print("RAM utilizada:")
print(gc.mem_alloc(), "bytes")

print()

print("Informações detalhadas:")
micropython.mem_info()
```

Prática 1 — Medindo a memória

Executarem o programa e observarem o **Serial Monitor**.

```
=====
      ORGANIZAÇÃO DA MEMÓRIA
=====

RAM livre:
...

RAM utilizada:
...

Informações detalhadas:
...
```

`gc.mem_free()`

retorna a quantidade aproximada de memória disponível para o gerenciamento de objetos

`gc.mem_alloc()`

informa a memória atualmente alocada para objetos gerenciados pelo Garbage Collector.

Prática 2 — Criando variáveis

```
import time
import gc
import micropython
```

```
numero = 123456
```

```
print("Depois de criar numero:")
print("RAM livre:", gc.mem_free())
print("RAM utilizada:", gc.mem_alloc())
```

```
import time
import gc
import micropython
```

```
texto = "Raspberry Pi Pico"
```

```
print("Depois de criar texto:")
print("RAM livre:", gc.mem_free())
print("RAM utilizada:", gc.mem_alloc())
```

```
import time
import gc
import micropython
```

```
lista = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

```
print("Depois de criar lista:")
print("RAM livre:", gc.mem_free())
print("RAM utilizada:", gc.mem_alloc())
```

Prática 3 — Lista grande

```
import time
import gc

print("RAM inicial:")
print(gc.mem_free())

dados = []

for i in range(1000):
    dados.append(i)

print()
print("Depois de criar 1000 elementos:")
print(gc.mem_free())
```

O que aconteceu com a memória livre?

A criação da lista aumenta o consumo de RAM.

Prática 4 - Visualizando a memória durante a execução

Prompt de Comando

```
Elementos: 0
-----
RAM utilizada: 6288
RAM livre: 239408
-----
Elementos: 100
-----
RAM utilizada: 6784
RAM livre: 238912
-----
```

```
import gc
import time

def mostrar_memoria():
    print("-----")
    print("RAM utilizada:", gc.mem_alloc())
    print("RAM livre:", gc.mem_free())
    print("-----")

print("Memória inicial")
mostrar_memoria()

dados = []

for i in range(0, 1000):
    dados.append(i)

    if i % 100 == 0:
        print("Elementos:", i)
        mostrar_memoria()
        time.sleep(1)
```

Prática 5 - Garbage Collector

Prompt de Comando

```
Memória inicial:  
239664  
  
Depois da lista:  
235536  
  
Depois de liberar a lista:  
241136
```

Por que a memória livre aumentou novamente?

```
dados = None
```

faz com que a variável deixe de referenciar a lista.

Depois:

```
</> Python
```

```
gc.collect()
```

solicita uma coleta de lixo.

O Garbage Collector identifica objetos que não são mais necessários e pode liberar a memória associada a eles.

```
import time  
import gc
```

```
print("Memória inicial:")  
print(gc.mem_free())
```

```
dados = []
```

```
for i in range(1000):  
    dados.append(i)
```

```
print()  
print("Depois da lista:")  
print(gc.mem_free())
```

```
dados = None
```

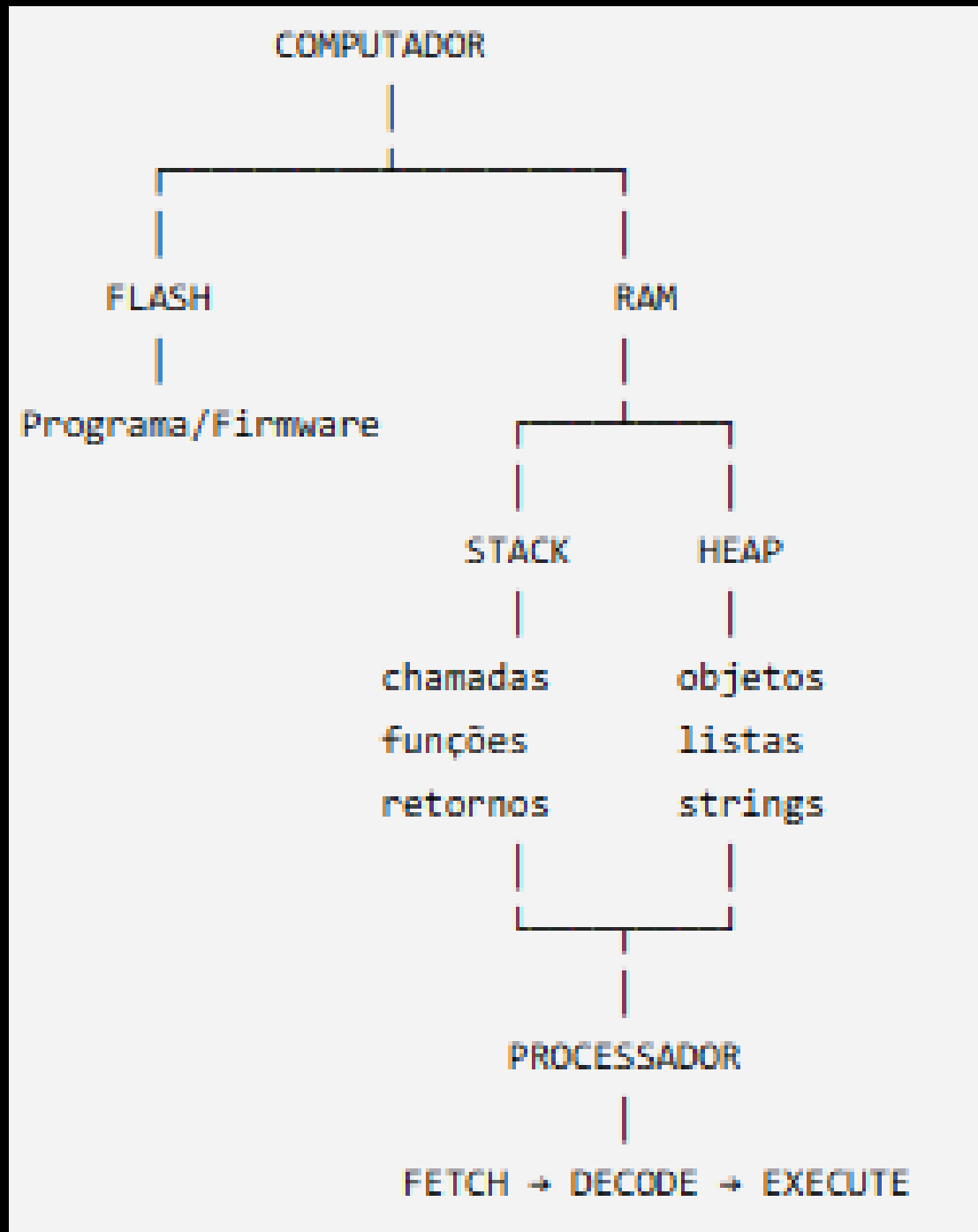
```
gc.collect()
```

```
print()  
print("Depois de liberar a lista:")  
print(gc.mem_free())
```

Resumo

Mapa Conceitual

O processador executa instruções, mas precisa da memória para armazenar as instruções, os dados e o estado necessário para que o programa funcione.



Atividade 1

"Detetives da Memória"

Criar um programa que:

- Mostre a RAM disponível inicialmente.
- Crie uma lista.
- Adicione 500 números.
- Mostre novamente a RAM.
- Crie uma segunda lista.
- Adicione mais 500 números.
- Mostre novamente a RAM.
- Apague a primeira lista.
- Execute `gc.collect()`.
- Mostre novamente a RAM.